



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



EJERCICIOS DE CLASE N° 08

NOMBRE COMPLETO: Velazquez Caudillo Osbaldo

N° de Cuenta: 320341704

GRUPO DE LABORATORIO: 02

GRUPO DE TEORÍA: 06

SEMESTRE 2026-1

FECHA DE ENTREGA LÍMITE: 21/10/25

CALIFICACIÓN: _____

EJERCICIOS DE SESIÓN:

1. Actividades realizadas. Una descripción de los ejercicios y capturas de pantalla de bloques de código generados y de ejecución del programa

1.- Agregar su dado de 8 caras y editar sus normales para que las caras del dado sean iluminadas correctamente.

Primero agregamos una variable de tipo Texture

```
49 Texture AgaveTexture,  
50 //Dado  
51 Texture dado_dosTexture;  
52
```

Agregamos la función para crear el dado a la cual le cambiamos los valores de las normales para que el dado pueda ser iluminado

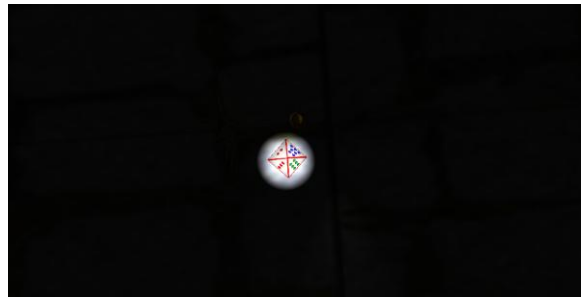
```
//dado 8 caras  
void CrearDado2()  
{  
    unsigned int cubo_indices2[] = {  
        // cara 0  
        0, 1, 2,  
        // cara 1  
        3, 4, 5,  
        // cara 2  
        6, 7, 8,  
        // cara 3  
        9, 10, 11,  
        // cara 4  
        12, 13, 14,  
        // cara 5  
        15, 16, 17,  
        // cara 6  
        18, 19, 20,  
        // cara 7  
        21, 22, 23  
    };  
    // average normals  
    GLfloat cubo_vertices2[] = {  
        // face 0 (top, front, right) 7 treboles azules  
        // x y z s t Nx Ny Nz  
        0.0f, 0.5f, 0.0f, 0.26f, 0.50f, -0.577350f, -0.577350f, -0.577350f,  
        0.0f, 0.0f, 0.5f, 0.51f, 0.75f, -0.577350f, -0.577350f, -0.577350f,  
        0.5f, 0.0f, 0.0f, 0.01f, 0.75f, -0.577350f, -0.577350f, -0.577350f,  
        // face 1 (top, right, back) 2-rosas rojas  
        0.0f, 0.5f, 0.0f, 0.74f, 0.50f, -0.577350f, -0.577350f, 0.577350f,  
        0.5f, 0.0f, 0.0f, 0.49f, 0.250f, -0.577350f, -0.577350f, 0.577350f,  
        0.0f, 0.0f, -0.5f, 0.99f, 0.250f, -0.577350f, -0.577350f, 0.577350f,  
        // face 2 (top, back, left) 3-corazones  
        0.0f, 0.5f, 0.0f, 0.74f, 1.00f, 0.577350f, -0.577350f, 0.577350f,  
        0.0f, 0.0f, -0.5f, 0.49f, 0.748f, 0.577350f, -0.577350f, 0.577350f,  
        -0.5f, 0.0f, 0.0f, 0.99f, 0.748f, 0.577350f, -0.577350f, 0.577350f,  
        // face 3 (top, left, front) 6-naipes verdes  
        0.0f, 0.5f, 0.0f, 0.50f, 0.25f, 0.577350f, -0.577350f, -0.577350f,  
        -0.5f, 0.0f, 0.0f, 0.75f, 0.505f, 0.577350f, -0.577350f, -0.577350f,  
        0.0f, 0.0f, 0.5f, 0.25f, 0.505f, 0.577350f, -0.577350f, -0.577350f,  
        // face 4 (bottom, right, front) 4-rosas-azules  
        0.0f, -0.5f, 0.0f, 0.74f, 0.50f, -0.577350f, 0.577350f, -0.577350f,  
        0.5f, 0.0f, 0.0f, 0.99f, 0.75f, -0.577350f, 0.577350f, -0.577350f,  
        0.0f, 0.0f, 0.5f, 0.49f, 0.75f, -0.577350f, 0.577350f, -0.577350f,  
        // face 5 (bottom, back, right) 5-naipes-verdes-reves  
        0.0f, -0.5f, 0.0f, 0.26f, 0.51f, -0.577350f, 0.577350f, 0.577350f,  
        0.0f, 0.0f, -0.5f, 0.01f, 0.255f, -0.577350f, 0.577350f, 0.577350f,  
        0.5f, 0.0f, 0.0f, 0.51f, 0.255f, -0.577350f, 0.577350f, 0.577350f,  
        // face 6 (bottom, left, back) 9-corazones rojas  
        0.0f, -0.5f, 0.0f, 0.255f, 0.01f, 0.577350f, 0.577350f, 0.577350f,  
        -0.5f, 0.0f, 0.0f, 0.50f, 0.26f, 0.577350f, 0.577350f, 0.577350f,  
        0.0f, 0.0f, -0.5f, 0.01f, 0.26f, 0.577350f, 0.577350f, 0.577350f,  
        // face 7 (bottom, front, left)  
        0.0f, -0.5f, 0.0f, 0.50f, 0.75f, 0.577350f, 0.577350f, -0.577350f,  
        0.0f, 0.0f, 0.5f, 0.25f, 0.50f, 0.577350f, 0.577350f, -0.577350f,  
        -0.5f, 0.0f, 0.0f, 0.75f, 0.50f, 0.577350f, 0.577350f, -0.577350f,  
    };  
};
```

Importamos la imagen que va a texturizar el dado

```
dado_dosTexture = Texture("Textures/dado_ocho_caras_optimizada.png");  
dado_dosTexture.LoadTextureA();
```

```
//Dado 8 caras  
model = glm::mat4(1.0);  
model = glm::translate(model, glm::vec3(-1.5f, 4.5f, -2.0f));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
glEnable(GL_BLEND);  
glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);  
dado_dosTexture.UseTexture();  
meshList[4]->RenderMesh();
```

Ejecución



2.- Apagar con teclado la luz (pointlight) de su lámpara creada para el reporte de la práctica 7.

Primero agregamos el modelo de la antorcha para usarlo en el código.

```
//iluminacion  
Model Antorcha_M;
```

```
Antorcha_M = Model();  
Antorcha_M.LoadModel("Models/antorcha.obj");
```

Creamos la luz puntual que se va a utilizar para la antorcha

```
//se crean mas luces puntuales y spotlight

//ANTORCHA
pointLights[0] = PointLight(
    1.0, 1.0, 1.0f,    // color: blanco
    0.2f, 0.9f,        // intensidades (ambient, diffuse)
    0.0f, 0.0f, 0.0f,  // posicion:
    0.3f, 0.1f, 0.1f   // atenuación
);
pointLightCount++;

//antorcha
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, 0.2f, -1.5));
modelaux = model;
pointLights[0].SetPos(glm::vec3(modelaux[3].x-0.1, modelaux[3].y+1.4f, modelaux[3].z));
model = glm::scale(model, glm::vec3(2.0f, 2.0f, 2.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Antorcha_M.RenderModel();
```

Agregamos la luz puntual por jerarquía a la antorcha con ayuda de SetPos

Podemos mover de ubicación nuestra luz puntual.

Para poder controlar la luz de la antorcha dentro de Window.cpp tenemos lo siguiente.

```
//para cambio
if (key == GLFW_KEY_O) {
    theWindow->cambiar = false;
}
if (key == GLFW_KEY_P) {
    theWindow->cambiar=true;
}
```

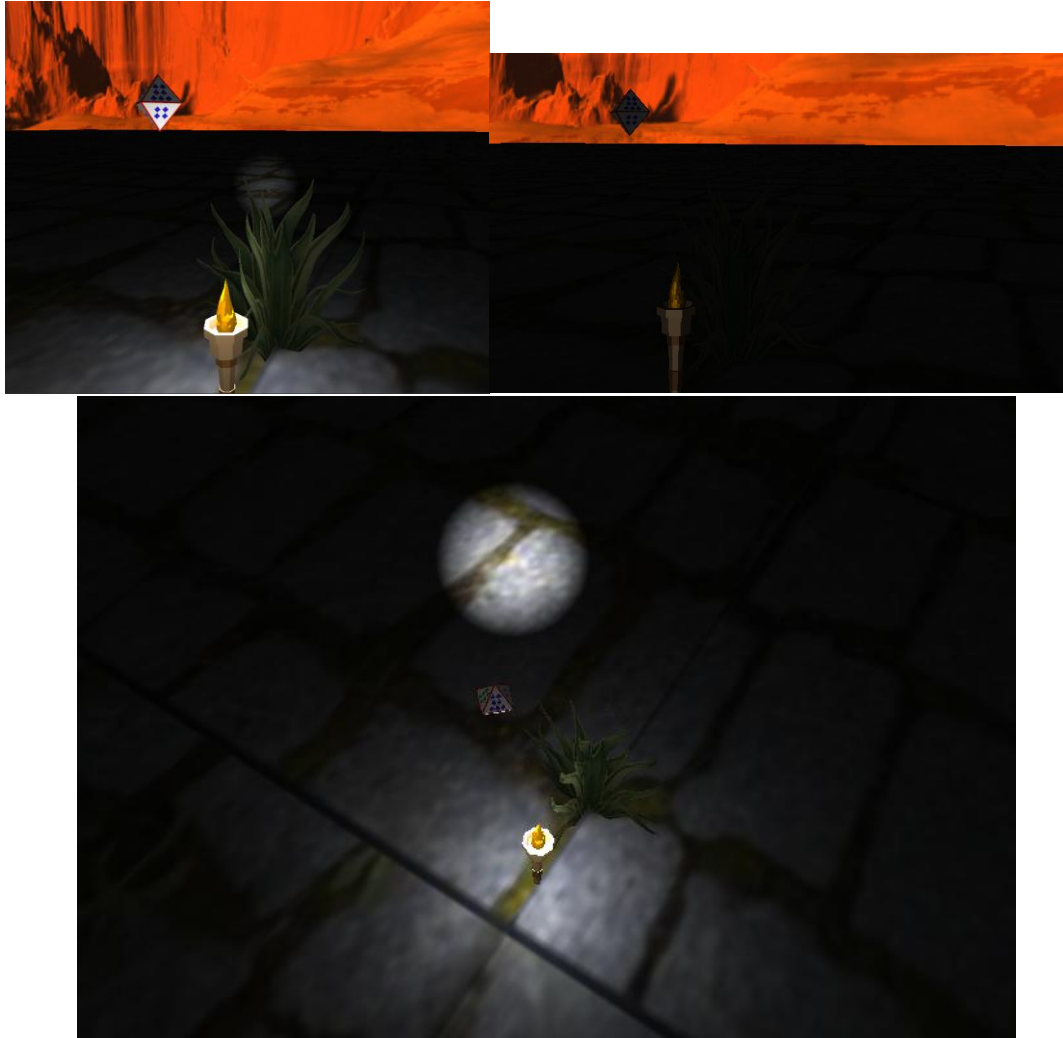
La tecla O la vamos a utilizar para apagar nuestra antorcha, mientras que encender la antorcha usamos la tecla P.

```
if (mainWindow.cambio()) {
    shaderList[0].SetPointLights(pointLights, pointLightCount);
    //printf("El valor de pointLightCount cuando prendo es: %d\n", pointLightCount);
    //printf("El valor booleano cuando prendo es: %d\n", mainWindow.cambio());
}
else {
    shaderList[0].SetPointLights(pointLights, pointLightCount - 1);
    //printf("El valor de pointLightCount cuando apago es: %d\n", pointLightCount-1);
    //printf("El valor booleano cuando apago es: %d\n", mainWindow.cambio());
}
```

Dentro del main lo que hacemos es que si el valor de la función cambio() es false entonces dentro del shaderList que trae a todos los pointLights al

contador le quitamos 1 para que no llame al pointlight de la antorcha y si es verdadero se pasa el valor original del pointLightCount para que mande el pointLight de la antorcha.

Ejecución



2. Problemas presentados. Listar si surgieron problemas a la hora de ejecutar el código

No se presentó ningún problema, durante la ejecución del código

3. Conclusión:

El ejercicio me ayudo a comprender y aprender como agregar luces con jerarquía, además de eso me ayudo a saber como agregar luces de manera eficiente en el código, para que no se este calculando durante todo el código si es que no se va a utilizar.

Este ejercicio fue de mucha ayuda para la interacción con las iluminaciones y su uso eficiente.